PBI AutoBuilder

Project structure: which folder contains what · React + Node.js (TypeScript) · raw data → Power BI project

```
pbi-autobuilder/
├── package.json        npm workspaces + scripts (dev / build / start / test)
├── shared/types.ts     data shapes shared by client and server
├── client/             React web app (the wizard UI)
├── server/             Node/Express API (all the real work)
├── samples/            demo data
├── scripts/            sample generators + end-to-end test
└── workspace/          your projects & settings (local only, never in git)
```

Top level

ITEM	CONTAINS
`package.json`	npm workspaces (client, server). `npm run dev` runs both; `build`, `start`, `test`.
`README.md`	Setup, usage and feature overview.
`.env.example`	Default AI provider settings (Gemini / Groq / Ollama) and `API_PORT`.
`.gitignore`, `.gitattributes`	Keeps `node_modules`, builds, `workspace/` and `.env` out of git; line-ending rules.
`shared/types.ts`	Every spec type: tables, sources, cleaning steps, model, measures, report pages, project.
`samples/`	`superstore_messy.csv`, `ecommerce_sales.csv`, `project_brief.md` for demos.
`scripts/`	`make-sample.mjs`, `make-ecommerce-sample.mjs` (generate messy data), `e2e.mjs` (full wizard run).
`workspace/`	Created at runtime: uploaded files, project JSON, exports, `settings.json` (API key). Private, not committed.

client/ — React front end

FOLDER / FILE	CONTAINS
`src/pages/`	One page per wizard step: **Home** (projects), **Upload** (files, scripts, SQL Server, file paths), **Profile** (understand data, KPIs), **Clean**, **Model**, **Measures**, **Dashboard** (multi-page preview), **Export**, **Settings** (AI provider).
`src/components/`	**Layout** (sidebar, Back/Next), **DataTable**, **OpEditor** (cleaning steps), **SchemaCanvas** (relationship diagram), **CodeEditor** (Monaco for M/DAX), **DashboardPreview** (charts), **DataSources** (SQL connect, path loader, scripts), **ui** (buttons, cards).
`src/api/client.ts`	All calls to the server API.
`src/store/projectStore.ts`	Current project state (Zustand).
Vite / Tailwind config	Build tool and styling setup.

server/ — Node.js back end

FOLDER / FILE	CONTAINS
`src/index.ts`	Express server on 127.0.0.1; serves the built client in production.
`src/routes/projects.ts`	API for each step: upload, profile, clean, scripts, SQL, model, measures, report, export, open in Power BI.
`src/routes/settings.ts`	Save AI settings, test the connection, list models.
`src/prompts/`	AI prompt templates: analyze, cleaning, model, measures, report.
`src/templates/README.md.hbs`	README written into every export.
`reference/`	Notes on the Power BI file formats.
`src/services/`	The engine (next section).

server/src/services/ — grouped by job

JOB	FILES
Reading data	`ingest.ts` (CSV/Excel), `sources.ts` (file paths, auto-load from scripts), `sqlserver.ts` (read-only SQL Server), `scripts.ts` (SQL/Python → cleaning steps), `profile.ts`, `values.ts`
Cleaning	`cleaning.ts` (every step, preview in JS), `m.ts` (Power Query M output)
Model & DAX	`model.ts` (relationships, Date table, ambiguity check), `dax.ts` (lint + evaluate)
Report	`report.ts` (layout of pages and visuals), `preview.ts` (numbers for the preview)
Suggestions	`rules.ts` (works without AI), `packs/collections.ts` (Collections & Recovery pack), `ai.ts` (AI suggestions), `llm.ts` (AI client, fallbacks), `schemas.ts` (validates AI JSON)
Export	`tmdl.ts` (semantic model), `pbir.ts` (report), `pbipWriter.ts` (writes the PBIP folder + zip), `opener.ts` (opens Power BI Desktop)
Support	`store.ts` (saves projects), `settings.ts`, `paths.ts`
Tests	`cleaning.test.ts` + `__snapshots__/` (Vitest)

How it flows

Upload → Understand → Clean → Model → Measures → Dashboard → Export .pbip → Save as .pbix

Each client page calls its route in `projects.ts`. The route asks `rules.ts` or the AI for a JSON spec, which you can review and edit. The export services then turn the specs into Power Query M, TMDL and PBIR files.

> **Principle:** AI decides, templates generate. The AI only proposes validated JSON; the Power BI files themselves are produced by tested code.